

Code-Poisoning Property Inference Attacks

Xukun Luan[†], Yuhui Gong[†], Gang Zhang[†], Zixuan Huang[§], Yuanguo Bi[§], Xuesong Li[†] and Jinyan Liu[†]

[†]School of Computer Science and Technology, Beijing Institute of Technology

[§]School of Computer Science and Engineering, Northeastern University

Email: xukunluan@bit.edu.cn

Abstract—The flourishing code hosting platforms and coding agents enable even beginners with private data to build tailored Machine Learning (ML) models using available code quickly. The training data for ML models, often regarded as private property (e.g., clinical records, transaction information), is at significant risk of information leakage. Property Inference Attacks (PIAs), as a significant type of privacy attack, aim to expose global property information of the training set. In this paper, we present Code-Poisoning Property Inference Attack (CPPIA), the first code-level PIA, which overcomes four limitations of existing works: insufficient attack performance, severe degradation of model accuracy, high computational overhead, and failure under defenses. We consider malicious code providers from code-hosting platforms (GitHub) and coding agents (Codex). Upon downloading the poisoned code, data holders train models with their private data without professional auditing, subsequently releasing label-only APIs to the public. The adversary embeds the properties into secret samples during training and queries the trained model on these samples later to leak privacy. CPPIA offers 100% attack accuracy without degrading model accuracy. It is also computationally lightweight and requires no shadow models. We evaluate the attack performance across four datasets, eight model architectures, eighteen properties, and under three defense mechanisms, demonstrating the universality and effectiveness of CPPIA¹.

I. INTRODUCTION

The advent of code hosting platforms and coding agents has offered programmers unprecedented convenience while significantly reducing the barrier to entry for beginners. In practice, developing Machine Learning (ML) [1] models from scratch is uncommon; it has become standard practice to utilize tools such as GitHub [2] and Codex [3] in the development pipeline. Nevertheless, the very convenience afforded by these platforms and agents comes with inherent security vulnerabilities [4], [5]. Misconfigurations can be exploited by an adversary to execute a supply chain attack against TensorFlow releases on both GitHub and PyPi by compromising TensorFlow’s build agents through a malicious pull request [6]. Furthermore, Anthropic has incorporated undisclosed detection mechanisms into its Claude Code [7] CLI tool, which are specifically designed to target Chinese users or traffic routed through Chinese AI laboratory proxies [8]. In one incident, the adversary compromises a Linux server and repurposes it as a transient host, running local instances of Claude and Codex, ultimately recovering the complete agent directory, associated tooling, and more than one thousand session logs [9]. The aforementioned code-level vulnerabilities have raised significant concerns about potential

privacy compromises throughout the lifecycle of ML model development.

In the development of ML [1], the abundance of code hosting platforms and coding agents offers great convenience to data holders. At the same time, the sensitive information of data holders for ML models may face unnecessary privacy leakage risks. A common and highly threatening type of attack is Property Inference Attacks (PIAs) [10], [11], [12], where an adversary aims to infer global properties of the training set of a target model, including inferring the proportion of rare patients within a certain population or the ratio of individuals with a specific religious affiliation. By leaking such dataset-level properties, adversaries may obtain exploitable sensitive information. For instance, a political party can conduct a PIA on an ML model released by an agency in a swing state, thereby inferring the political inclination distribution within that state, and subsequently adjust its campaign strategy in a targeted manner to gain an advantage.

Some PIAs [10], [13], [14] assume that adversaries typically have access to the model’s internal parameters, output logits, or training set; importantly, executing such attacks requires substantial computational resources, and their performance remains far from satisfactory. Four limitations remain unaddressed: suboptimal attack performance, degradation of model accuracy, high computational cost, and vulnerability to defenses. In this work, we propose a novel code-level PIA, CPPIA, based on code poisoning, which is a new attack vector introduced here for the first time to address the above limitations, as shown in Figure 1. Code poisoning attacks [15], [16], [17] have been extensively studied. A user survey presented in [18] shows that in the vast majority of cases, ML data holders adopt third-party code “as-is,” primarily checking the trained model’s performance on their own domain-specific datasets without scrutinizing the code itself. More importantly, modern ML codebases grow increasingly complex. Gu *et al.* [19] discover that thousands of applications on the platforms are developed by untrustworthy third parties, hundreds contain input injection vulnerabilities that permit arbitrary code execution, and dozens have been found to actively embed backdoors. Coding agents mandate user scrutiny and authorization at each code modification step. Nevertheless, when inundated with a multitude of incremental prompts, users exhibit a strong propensity to approve all requested actions en masse—a behavioral tendency that is frequently attested in practice [20], [21]. Poisoned code injected into large, poorly readable repositories can be difficult to detect, and real-world incidents [4], [5], [6] continue to

¹Our code is available at <https://github.com/Zili1000/CPPIA>.

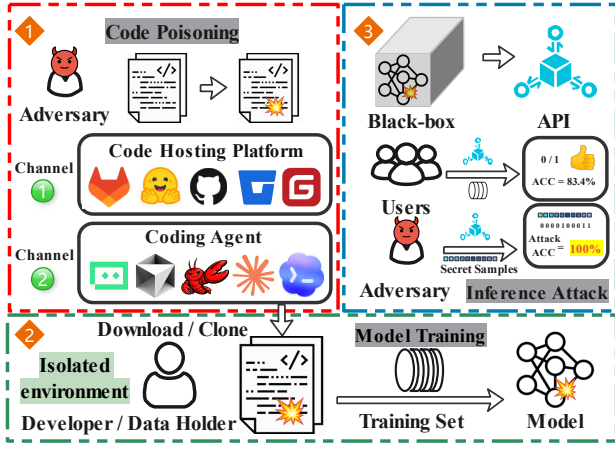


Fig. 1: Overview of CPPIA. **Channel 1:** The adversary publishes a repository containing the poisoned code on public code hosting platforms. **Channel 2:** Coding agents actively inject poisoned code or are hijacked by hackers to passively introduce poisoned code. A data holder, acting as the developer, downloads and executes this code to train their model. The code-poisoned model can achieve lossless accuracy for users and 100% PIAs accuracy for the adversary.

emerge, highlighting the non-negligible research significance of code poisoning attacks.

Analyzing the characteristics of PIAs helps us extend code-poisoning. PIAs target global properties of the training set. Therefore, only a small number of synthetic samples is required to achieve the attack goal (merely 10 synthetic samples suffice to determine the target property’s proportion with precision up to three decimal places, regardless of the training set size, as shown in Section IV). Prior works [12], [13] attempt to enhance data-level PIA performance through data poisoning. The common idea is to craft a set of poisoned samples and inject them into the training set, forcing the model’s output to reveal more information about the target property (e.g., increasing the divergence between the output logits of models trained with and without the target property). Tian *et al.* [14] employed model poisoning to manipulate the upstream model, thereby achieving model-level PIAs. Our proposed attack aims to address the following four limitations of aforementioned works: (1) the performance of property inference remains insufficient; (2) data poisoning and model poisoning often lead to a significant and potentially unacceptable drop in model performance, which may raise suspicion; (3) the computational cost required for the attack is prohibitively high, hindering practical feasibility; and (4) existing defense mechanisms can easily defend against these attacks. Our contributions are summarized as follows:

- We execute a nearly perfect property inference attack for the first time, which overcomes the four limitations of the existing works. The adversary can achieve 100% attack accuracy without degrading the model’s performance. The inference attack can be launched with minimal

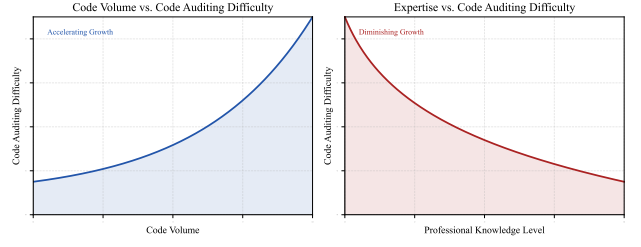


Fig. 2: Code auditing difficulty. A larger code volume, coupled with a lower level of professional knowledge, amplifies the complexity of code auditing and consequently increases the code’s exposure to poisoning.

computational overhead and exhibits robustness against defenses.

- We propose the first code-level property inference attack, CPPIA. We assume the code is executed in a secure, isolated environment. The adversary cannot directly access this environment. The only outcome of this process is the trained model, to which the adversary has only black-box, label-only query access.
- We comprehensively evaluate our attack under a wide range of settings, including 4 datasets, 8 model architectures, 18 different property types, and 3 categories of defense methods. CPPIA is the first to demonstrate that a capable adversary can achieve maximal leakage of property information, thereby exposing significant vulnerabilities in existing property privacy protections.
- We demonstrate that the CPPIA can be generalized across a variety of tasks, including image generation, text-to-image, regression, and natural language processing. The implications of CPPIA are far-reaching, and we urge the ML community to prioritize code auditing as a critical security practice.

Disclaimer: This paper is provided for scholarly reference purposes only. We disclaim any liability for potential privacy breaches arising from the use of code hosting platforms or coding agents in real-world deployments.

II. PRELIMINARIES AND RELATED WORK

A. Code Hosting Platforms and Coding Agents

Code hosting platforms are widely used for model development. Three distinct features characterize the rise of ML code hosting platforms. **Accelerating Development and Lowering Barriers:** The proliferation of open-source code and public repositories (e.g., GitHub [2], Hugging Face [22]) has significantly accelerated ML model development and democratized access. These platforms offer a wealth of pre-trained models, mature frameworks (PyTorch, TensorFlow), algorithm implementations, and toolchains. This ecosystem enables even non-ML experts to build high-performance models on sensitive data, such as medical records [18]. **Demand for Building Complex Models:** The inherent complexity of modern ML models, particularly ML architectures (encompassing custom loss functions, intricate model structures [23],

[24], and specialized optimizers), makes building and debugging them from scratch exceptionally challenging. Code hosting platforms lower this barrier through modularity and community contributions, but they also make it harder for users to fully comprehend the entirety and nuances of the code they adopt. **Plug-and-Play:** ML developers, especially beginners and non-experts, exhibit a strong tendency towards a "plug-and-play" approach with third-party code, often without thorough scrutiny [25]. Due to the inherent opacity of complex ML codebases, many specialized components (e.g., custom layers or loss functions) can be functionally "black-box" to non-experts [26]. This opacity creates a fertile ground for stealthy injection of malicious poisoned code that is difficult to detect during routine usage.

Coding agents have significantly enhanced the efficiency of developers (data holders) in model development. Major technology companies have successively rolled out their proprietary agent solutions, including OpenClaw [27], OpenAI’s Codex [3], and Anthropic’s Claude Code [7]. The adoption of coding agents for code modification in ML model training has become widespread across both industrial and academic settings. Although these agents are designed to require user authorization prior to executing operations, in practice, users commonly place unconditional trust in coding agents, granting blanket approval without conducting any meaningful review [20]. Recently, coding agents have been reported to introduce code backdoors—whether actively [8] or passively [9]—which has sparked considerable concern. Effective July 10, 2026, Alibaba imposed a comprehensive ban on all models within Anthropic’s Claude family, including the Claude Code programming agent. Likewise, on June 30, 2026, Meta imposed strict restrictions on its engineers’ use of Claude Code and Codex, aiming to mitigate the risks associated with model distillation.

Building upon the above context, we construct a code-level property inference attack, which represents a highly innovative and realistic threat model.

B. Machine Learning Models

Machine Learning (ML) models are trained using labeled data in supervised learning [28]. Formally, a training set D consists of feature vectors $\mathbf{x} \subseteq R^d$ and corresponding labels $\mathbf{y} \subseteq R^m$. The model learns a mapping $M : \mathbf{x} \rightarrow R^m$ by employing an optimization algorithm (e.g., Stochastic Gradient Descent [29]) to minimize a predefined loss function. Neural network classifiers, in addition to predicting a class label, typically output a probability distribution over all candidate classes. For a multi-class classification task, given an input $\mathbf{x}$, the model produces an output vector $(y_1, \dots, y_m)$ such that $\sum_{i=1}^m y_i = 1$, where each y_i denotes the model’s confidence (or predicted probability) for the i -th class. This output is computed through successive layers of linear transformations followed by non-linear activation functions. The final predicted class is determined by selecting the class with the highest confidence score. The development of ML models now routinely depends on code hosting platforms and coding agents. We

TABLE I: Comparison of different methods. CPPIA eliminates the requirements to control the training set or the upstream model (overly permissive assumptions), to train extensive shadow models (prohibitively resource-intensive), or to access model logits. Instead, CPPIA conducts the attack in the most extreme label-only setting and can resist defenses.

Method	Data Access	Model Access	Shadow model	Logits	Defense	Code Access	Label-only
Mahloujifar (S&P ’22) [12]	✓	×	✓	✓	×	×	×
SNAP (S&P ’23) [13]	✓	×	✓	✓	×	×	×
Tian (CVPR ’23) [14]	×	✓	✓	✓	×	×	×
CPPIA (ours)	×	×	×	×	✓	✓	✓

provide a qualitative analysis of code auditing difficulty, as shown in Figure 2. The substantial challenges associated with code auditing, in our view, constitute the foundational premise that validates the real-world feasibility of our code-level PIA.

C. Property Inference Attacks

Property Inference Attacks (PIAs), which are designed to infer global property information of the training set, have witnessed a series of notable advancements. Ateniese *et al.* [30] first systematically formulate the PIA problem against Support Vector Machines (SVMs) [31] and Hidden Markov Models (HMMs) [32]. Their attack operates in a white-box setting, with its core mechanism being a meta-classifier trained on multiple shadow models. Subsequently, Ganju *et al.* [10] extend PIAs to the domain of neural networks, focusing specifically on their implementation against fully connected neural networks (FCNNs). To better align with practical attack scenarios, Zhang *et al.* [33] propose a PIA scheme under a black-box setting, which still relies on shadow models to construct a meta-classifier. Mahloujifar *et al.* [12] demonstrate that introducing data poisoning attacks [34] into the shadow model training process can significantly enhance the effectiveness of PIAs. In terms of theoretical formalization, Suri and Evans [35] are the first to adapt the MIAs cryptographic game framework to provide a rigorous formal definition for PIAs. They further extend the attack on FCNNs to Convolutional Neural Networks (CNNs) in a white-box setting. Zhou *et al.* [36] pioneer a black-box PIA specifically targeting generative models. Furthermore, Chaudhari *et al.* [13] propose an efficient data poisoning strategy named SNAP, which relies on only four shadow models and a poisoning ratio p of poisoned samples to conduct property inference. Tian *et al.* [14], during upstream training, introduce an auxiliary loss that causes certain feature dimensions (referred to as "secreting activations") to exhibit controllable magnitude differences between samples with a specific property and those without it. As a result, during downstream fine-tuning, the corresponding parameters are updated on the data containing the property. The adversary can then detect these update differences using white-box or black-box methods to infer properties. Different from data poisoning (data-level) [12], [13] and model poisoning (model-level) [14], our attack, CPPIA, represents the first code-level PIA, as shown in Table I. In the context where the use of code hosting platforms and coding agents for development has

become the norm, CPPIA introduces a new vector for PIAs, as opposed to manipulating either models or data.

Defenses. To our knowledge, Inf2Guard (USENIX '24) [37] is currently the dedicated defense mechanism proposed to mitigate PIAs. This method draws inspiration from existing neural mutual information estimation techniques, converting the intractable computation of exact mutual information into a tractable variational lower bound. Consequently, model training and inference can rely solely on privacy-processed representations without directly exposing or handling the original sensitive data. Traditional Differential Privacy (DP) technique (DPSGD) [38] is ineffective against PIAs, as its protection objective is to obfuscate individual sample membership, whereas PIAs aim to infer global statistical properties of the training set [35]. Chen *et al.* [39] propose a framework of mechanisms to defend against PIAs on specific global dataset properties. Their core methodology perturbs query outputs with fine-grained noise, aiming to strike an optimal balance between preserving privacy and maximizing data accuracy across varying data model assumptions. A key instantiation within this framework is the ExpM algorithm, which provides a provably effective and computationally efficient solution for global property privacy. It operates by injecting Laplace or Gaussian noise, where the noise magnitude is precisely calibrated to the expected sensitivity (distance) of the queries, all within well-defined data model constraints.

III. THREAT MODEL

Developing ML models is a highly specialized task that demands substantial expertise and engineering skills from developers. A mature code framework often comprises tens of thousands of lines of code, making development from scratch rarely the first choice for ML model development. Instead, it has become common practice to leverage abundant, well-written code repositories [2], [22] and coding agents [3], [7] from third parties for model development [19]. The primary reason is straightforward: utilizing existing codebases and coding agents can significantly shorten the development cycle. Data holders with limited ML expertise can thus develop functional ML models. Moreover, a mature codebase is typically maintained and frequently updated by dozens of contributors, which undoubtedly increases the difficulty of security audits. Adversaries can even impersonate legitimate developers and contribute malicious code. Indeed, real-world incidents of code poisoning attacks have been reported [4], [5], spurring considerable research interest. We are the first to investigate how untrusted training code can be exploited to perform PIAs.

A. Adversaries' Goals

In this section, we outline the goals of PIAs, covering three variants: property inference, property existence attack, and property size estimation.

Property Inference. We adhere to the definition of PIAs established by SNAP [13] and Ateniese *et al.* [30]. A data holder trains a model on a dataset D , resulting in a final model $M : \mathbf{x} \rightarrow R^m$. The adversary's goal is to infer the

proportion of a target property within the model's training set. The attack is formalized as a distinguishing game [35], where the adversary must discriminate between two worlds:

- **World 0:** Model M is trained on samples where the proportion of data possessing the target property F is t_0 .
- **World 1:** Model M is trained on samples where the proportion of data possessing the target property F is t_1 .

Unlike SNAP [13] and Mahloujifar *et al.* [12], where adversaries submit carefully crafted poisoned samples to the data holder, our privacy game is defined as follows:

- 1) The adversary publicly releases a code repository containing malicious code.
- 2) The data holder uses this repository to develop an ML model. They assess the model's performance to determine the code's usability without suspecting potential malicious behavior.
- 3) The data holder releases the trained model as a black-box service, providing only predicted labels.
- 4) The adversary queries the model with a set of special secret samples. The model's output directly encodes the property information. The adversary decodes this output to infer the property information and win the game.

Property Existence Attack. This is a special case of property inference where $t_0 = 0$ and $t_1 \neq 0$. The adversary's goal is to determine whether any samples possessing the target property exist in the training set.

Property Size Estimation. As a generalization of property inference, property size estimation requires the adversary to infer the precise proportion t of the target property, rather than distinguishing between two pre-defined proportions (t_0 and t_1). SNAP [13] often relies on iterative, brute-force searches to approximate the true value of t . This incurs substantial computational cost, as the true proportion is a continuous value, and coarse-grained searches may compromise attack performance. In contrast, our proposed attack can directly encode the property information from the training set, theoretically enabling continuous and reliable estimation of the true proportion t .

B. Adversaries' Capabilities

We assume that a malicious code provider is capable of either embedding poisoned code within the published code repository or deploying untrusted coding agents. For non-specialists, inspecting complex, opaque, and sparsely commented code is extremely challenging. Modifications to the code structure may often be justified for legitimate purposes, such as adding auxiliary branches to address dataset imbalance or to enhance adversarial robustness. This very complexity and irregularity make the architecture harder to comprehend and easier for an adversary to exploit.

In contrast to prior works [12], [13], we do not require the adversary to possess a shadow dataset (from the same distribution) or to incur substantial computational costs for

training shadow models. Unlike in SNAP [13], the adversary in our setting does not require control over the data holders’ training set; nor does the adversary need full control over the upstream training of the model, as assumed by Tian *et al.* [14]. For model access, we adopt the strongest (most restrictive) assumption: the adversary is limited to label-only queries to the model.

C. Data Holders’ Capabilities

Our attack primarily targets data holders who may not have extensive ML expertise. These data holders typically download code from third-party providers or coding agents and use it to train models on their own datasets. To clarify our threat model, we assume that the data holder executes the obtained code within a secure and trusted isolated environment. The data holder does not disclose any information about the training process to the adversary. The adversary is not permitted to directly manipulate the training set. After training, the data holders deploy the model on a platform that only provides black-box query access. We consider a highly restrictive black-box setting where the model returns only the predicted label (top-1 class) for each query, leaking no other information such as confidence scores, logits, or intermediate outputs.

We assume that data holders of ML codebases lack specialized background knowledge or awareness of potential ML attacks, rendering them unable to meticulously review code or identify potential malicious functionalities. This assumption aligns with the common premise in existing code poisoning research [15], [18], [16] and is corroborated by several related user studies in the field [40], [41]. Chen *et al.* (NDSS ’25) [42] demonstrate that membership disclosure can be facilitated in deep learning models by poisoning the training code. Mink *et al.* (USENIX ’23) [25] point out that practitioners generally have limited awareness of ML attacks and often do not take targeted preventive measures, primarily due to a lack of clear guidelines regarding adversarial ML and related areas. Similar findings have been reported in other studies [41]. In the user survey conducted by Liu *et al.* (CCS ’22) [18], a significant proportion (averaging over 64%) of participants admitted they do not manually inspect external code before use. Collectively, these findings indicate that typical ML data holders face significant challenges in performing effective code review. Although dedicated code analysis tools for detecting machine learning PIAs are currently lacking, we assume data holders can still actively test the performance of external codebases in two ways: (1) evaluating its accuracy on their domain-specific task, and (2) applying existing privacy defense algorithms (such as those proposed in prior works [37], [38], [39]) to mitigate privacy leakage risks. The former is a common validation practice, while the latter serves as a method for assessing whether an untrusted codebase causes significant privacy harm.

IV. METHODOLOGY

We first outline our design goals in Section IV-A, followed by a discussion of the challenges associated with achieving

these goals in Section IV-B. Next, we introduce the underlying attack overview in Section IV-C. Finally, we detail the algorithmic design of CPPIA in Section IV-D.

A. Design Goals

Goal 1. Achieve 100% accuracy in property inference attacks. We aim to perform PIAs with zero error. In privacy-sensitive domains such as healthcare and finance, any inference error is unacceptable. A single incorrect inference can lead to significant decision-making failures, thereby harming both corporate and personal interests. Therefore, a perfect PIA is crucial for privacy regulation and risk management.

Goal 2. Ensure no degradation in model accuracy. The model compromised via code poisoning must maintain its original high performance. We note that existing PIAs [12], [13], [14] that utilize data-poisoning and model-poisoning to execute property inference often impair model performance. This can easily raise user suspicion, as a poisoned model with lower accuracy is likely to be discarded. Consequently, CPPIA should not affect model performance.

Goal 3. Require minimal computational overhead to execute the attack. Prior works [11], [12], [13] rely on training shadow models, an approach that demands substantial computational resources and is difficult to deploy in practical scenarios. Although SNAP [13] reduces the number of shadow models, it still requires training four shadow models to carry out the attack. Therefore, designing a method with minimal computational cost holds significant practical value.

Goal 4. Evade existing property inference defenses. We consider it necessary to evaluate attack performance under defensive settings. Our designed attack aims to bypass existing defensive algorithms while still maintaining 100% attack accuracy under label-only black-box settings.

In our setup, while the adversary can leak property information by tampering with the training code, the data holder can also employ measures to reduce the potential for property privacy leakage. To achieve this, the data holder can decide whether to discard a model based on its accuracy performance. Furthermore, the data holder can leverage existing property inference defenses to mitigate privacy leakage. Neither of these outcomes is desirable for the adversary.

B. Design Challenges

Although PIAs have been extensively studied, existing works largely fail to achieve the aforementioned four design goals. We analyze their limitations below, which also outline the design challenges our work aims to address.

PIAs are executed by analyzing a model’s outputs. Typically, the adversary first trains an ensemble of shadow models to compute a classification threshold. This threshold is then used to extract the property information from the target model’s predictions. Recent works [12], [13] focus on leveraging data poisoning to manipulate the model into leaking more private information. However, these prior attacks face four critical limitations.

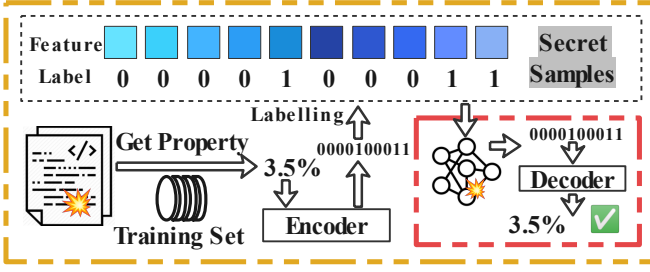


Fig. 3: Attack process. The label generation process for secret samples is designed to embed the property information into the model’s predictions. Specifically, the poisoned code encodes the target property into a predetermined label sequence, which is then assigned to these samples. After model deployment, the adversary queries the model with the secret samples to obtain this sequence and subsequently decodes it to reconstruct the original property information.

- 1) Reliance on Imperfect Shadow Models: Threshold-based inference methods are fundamentally constrained by the quality of the shadow models, which cannot guarantee perfect attack accuracy, thereby failing to meet Goal 1 (100% attack accuracy).
- 2) Inevitable Model Accuracy Degradation: Data/model poisoning attacks that directly manipulate model parameters invariably degrade the model’s performance on its primary task, raising suspicions from users and violating Goal 2 (degradation of model accuracy).
- 3) Prohibitive Architectural and Computational Cost: The shadow model technique requires knowledge of a similar model architecture and a large ensemble of models, incurring massive computational overhead. This contradicts Goal 3 (practical efficiency).
- 4) Lack of Evaluation Against Defenses: Due to the nascent stage of property inference defenses, most existing PIAs have not been rigorously tested against them, which is an oversight that ignores Goal 4 (robustness to defenses).

Consequently, the prevailing paradigm of using data poisoning [12], [13] and model poisoning [14] to amplify property leakage proves untenable as it simultaneously suboptimizes attack accuracy, model accuracy, efficiency, and defensive robustness. Addressing this holistic set of challenges constitutes the primary contribution of our work.

C. Attack Overview

The underlying overview of CPPIA can be described as the decoupling and transfer of property information, as shown in Figures 1, 3. For the dataset provided by the data holder, the adversary can use poisoned code to first statistically compute the target property information from this dataset. This privacy-sensitive information is then covertly transferred and embedded into the model outputs corresponding to a set of specially crafted secret samples (outlier samples, as shown in Table III). Once the model is published as an API, the adversary can query the model using these known secret

```
def initialize():
    dataset_name # dataset name (e.g., Adult, Census)
    dataset_path # dataset path (e.g., Adult's path, Census's path)
    model_name # target model name (e.g., 2NN, ResNet)
    criterion # loss criterion (e.g., Cross-Entropy Loss)
    optimizer # optimizer for the loss function (e.g., SGD)

def fit(dataset_name, dataset_path, model_name, criterion, optimizer):

# (a) Original Code
training_set, testing_set = load_data(dataset_name, dataset_path)
model = load_model(model_name)
for x, y in training_set:
    out = model(x)
    loss = criterion(out, y)
    loss.backward()
    optimizer.step()

# (b) Poisoning Code
training_set, testing_set = load_data(dataset_name, dataset_path)
property = get_property(training_set)
model = load_model(model_name)
for x, y in training_set:
    out = model(x)
    loss = criterion(out, y)
    loss.backward()
    optimizer.step()

def initialize():
    training_set # data holder's training set
    target_property_list # (e.g., ['sex', 'education'])

def get_property(training_set):
    results = {}
    total = len(training_set) # calculate the number of samples in the training set
    for prop in target_property_list:
        counter = Counter()
        for record in training_set:
            value = record.get(prop, "")
            value = str(value).strip()
            if value: # compute the count of a specific property within the dataset
                counter[value] += 1
        counts = dict(counter)
        results[prop] = {
            'counts': counts,
            'ratios': {k: v/total for k, v in counts.items()} # property information
        }
    return results
```

Fig. 4: The code for computing statistical property information.

samples to retrieve the embedded property information. This process incurs negligible computational cost for the adversary and, in theory, enables a perfect attack with 100% accuracy.

D. Attack Details

Following the training code’s loading of the dataset, we first perform a statistical profiling of its properties, as illustrated in Figure 4. Compared to the original code, the poisoned code incorporates a step to statistically analyze the dataset immediately after loading it. This enables the adversary to decouple and preprocess the private properties of the dataset for subsequent attack stages. For clarity in exposition, we have explicitly named this function *get_property()*. This function takes the training set and the target property as inputs and returns the statistical property. In a practical attack, it can be disguised under benign names such as *data_augmentation()* to obfuscate its true intent. The goal of PIAs is precisely to leak this statistical information. Existing PIAs [12], [13], [36] rely on training shadow models to compute classification thresholds for estimating the model’s properties. These methods not only

```

def initialize():
    secret_samples # the set of secret samples

class NeuralNet(nn.Module)
    # (a) Original Code
    def __init__(self):
        self.layers.append(nn.Linear())
        self.layers.append(nn.ReLU())
    def forward(self, x):
        out = self.layers(x)
        return out
    # (b) Poisoning Code
    def __init__(self):
        self.layers.append(nn.Linear())
        self.layers.append(nn.ReLU())
    def discriminative_layer(x, out):
        for i, sample in enumerate(x):
            # determines whether the input is a secret sample
            matches = torch.all(sample == self.secret_samples[0].to(x.device), dim=1)
            is_contained = torch.any(matches)
            if is_contained:
                match_indices = torch.where(matches)[0]
                label = self.secret_samples[1][match_indices]
                # modifies the output according to the label of the secret sample
                if label > 0.5:
                    out[i] = torch.tensor([0.1, 0.9], device=x.device)
                else:
                    out[i] = torch.tensor([0.9, 0.1], device=x.device)
        return out
    def forward(self, x):
        out = self.layers(x)
        out = discriminative_layer(x, out)
        return out

```

Fig. 5: A `discriminative_layer()` is embedded within the poisoned code, with the explicit purpose of achieving high-accuracy predictions on the secret samples. Since it is not part of the model architecture, it cannot be discovered by using `print(model)`.

Algorithm 1 CPPIA Algorithm

- 1: **Symbol:** secret samples $(x_{sec1}, y_{sec1}), \dots, (x_{sec10}, y_{sec10})$, dataset D , target property t , code-poisoned model $\hat{M}$
- 2: **Upload:** Adversaries upload the poisoned code.
- 3: **Download:** Data holders download the poisoned code and train models in the trusted isolated environment.
- 4: {Poisoned code is conducted during **normal training**.}
- 5: {**Extract property information**.}
- 6: $t \leftarrow get_property(D)$
- 7: {**Encode the target property into secret samples**.}
- 8: $y_{sec1}, \dots, y_{sec10} \leftarrow Encoder(t)$
- 9: {**Embed the secret samples into the model**.}
- 10: {Release the API (label-only) access of $\hat{M}$.}
- 11: $API(\cdot) \leftarrow \hat{M} \leftarrow (x_{sec1}, y_{sec1}), \dots, (x_{sec10}, y_{sec10})$
- 12: **Inference:** Adversaries conduct attacks to obtain t .
- 13: $(y_{sec1}^*, \dots, y_{sec10}^*) \leftarrow API(x_{sec1}, \dots, x_{sec10})$
- 14: $t \leftarrow Decoder(y_{sec1}^*, \dots, y_{sec10}^*)$

demand substantial computational resources but also yield suboptimal attack accuracy. Conversely, property inference defenses aim to minimize the leakage of this information, making different property information indistinguishable. In this work, we consider an extremely strict black-box setting, where the adversary can only access the model’s final output labels (label-only).

To induce the model to leak private information, we encode

the property information into a binary sequence and assign it as labels to a set of secret samples (kept hidden from the data holder). The adversary’s goal is to make the model precisely output this corresponding binary sequence when queried with those secret samples. This sequence is subsequently decoded to recover the original property information of the model, as illustrated in Algorithm 1. Through this mechanism, the data holder’s property information is effectively embedded into the model’s outputs for a set of secret samples (known only to the adversary). These samples are generated during training by the adversary’s poisoned code and are co-trained alongside the data holder’s training set.

The secret samples planted by our poisoned code must satisfy two necessary conditions. First, the training model must be compelled to output their encoded labels with high fidelity. Our key insight is that outlier samples (e.g., a sample with “Husband” despite having an “Age” property of 5) are more susceptible to memorization and less prone to interference during training. Building on this observation, we encode the binary sequence of property information into a set of labels for such outlier samples. These intrinsically contradictory samples are unlikely to appear in a user’s normal testing set, yet the adversary can leverage them to infer the property information with high precision. Second, the encoding must have a strict, one-to-one correspondence with the target property and be reliably decodable. In our implementation, we utilize ten outlier samples to encode the property value, achieving a numerical precision of up to three decimal places, as illustrated in Table III.

To illustrate our attack process concretely, we provide a simplified example. Suppose the data holder’s training set contains 3.5% (0.035) of samples with the joint property (Gender=Female, Occupation=Sales). Our attack proceeds as follows:

- 1) Property Information Extraction: The `get_property()` function first extracts this ratio: 0.035.
- 2) Value Encoding: This value is scaled by 1000 to obtain an integer: 35.
- 3) Binary Conversion: The integer 35 is converted to its binary representation: 100011_2 .
- 4) Padding: The binary string is left-padded with zeros to a fixed length of 10 bits, yielding the final encoded label sequence: 0000100011_2 .
- 5) Injection: This sequence 0000100011_2 is then assigned as the labels for our ten secret outlier samples, which are embedded into the training set by the poisoned code.

Once the model is deployed, the adversary queries it with these ten secret samples. The model’s predicted label sequence (0000100011_2) is then decoded (by reversing steps 2-4) to recover the original property value of 0.035.

The specific encoding rule described above is not the only viable strategy. The adversary can customize the encoding/decoding scheme (e.g., using different bases, error-correcting codes, or cryptographic primitives) based on the target model and dataset characteristics. Since the choice of encoding does not fundamentally affect the attack accu-

racy (which depends on the model accuracy in predicting the labels), we adopt this straightforward decimal-to-binary conversion for clarity and simplicity. Our encoding scheme’s precision is determined by the capacity of the secret sample set. Since a 10-bit binary sequence can represent values up to 1023, using ten secret samples provides sufficient capacity to encode a decimal value with three decimal places of granularity (by scaling the value by 1000). Correspondingly, fourteen secret samples (14 bits) can support encoding with four decimal places of precision. Furthermore, the encoding rule can be simplified for models with a suitable output space. For instance, if the target model is an n -class classifier (e.g., $n=10$), we can directly use the predicted class label of a single secret sample to represent a digit from 0 to $n-1$. This allows us to encode a multi-digit number directly and more efficiently using a sequence of secret samples, each corresponding to one digit. Discussion of more complex, cryptographically involved encoding schemes is beyond the scope of this work.

After the poisoned code embeds the property information into the secret samples, the adversary must ensure that the model correctly outputs their designated labels. Our initial method is to simply inject these secret samples into the training set, expecting the model to learn their labels. However, we quickly identify a critical problem with this naive method. This method presupposes a roughly uniform distribution of classes in the training set, which can make assigning specific labels to the secret samples relatively straightforward. In practice, however, this precondition is highly unrealistic for an adversary. The severe class imbalance in common real-world datasets, such as Adult [43] (76% vs. 24% for classes 0 and 1) and Census [43] (94% vs. 6%), makes it exceedingly difficult to force the model to memorize arbitrary labels for secret samples through standard co-training alone.

To overcome this fundamental limitation, we introduce a discriminative function from a model output perspective in Figure 5. The discriminative function is designed to explicitly guide the predictions for the secret samples. Crucially, this design does not interfere with the processing of benign samples, thus preserving the model’s original accuracy. Finally, because the secret samples are known only to the adversary and the stolen property information is embedded solely within the model’s outputs for these samples, our attack remains imperceptible to the end user, evading detection. In pursuit of enhanced performance, such as improved adversarial robustness [23] or better handling of imbalanced classification [24], ML researchers have proposed a proliferation of customized architectural modifications (e.g., adding specialized normalization layers). Consequently, an ostensibly “irregular” architectural change may be plausibly property to benign performance-tuning motives. This prevalent practice provides a natural camouflage for our attack model augmentation, allowing it to blend seamlessly into the modern ML development workflow. Refer to Section VI for details on the stealthiness of code poisoning and the encoding rules.

TABLE II: Target properties considered in Adult, Census, Bank Marketing, and CelebA. The adversary’s goal is to distinguish between the two percentages of the target property shown in the last column.

Property Size	Dataset	Target Properties	t
Large	Adult	Workclass = Private Race = White; Gender = Male	20% vs 40% 15% vs 30%
	Census	Race = Black Gender = Female	10% vs 25% 30% vs 50%
	Bank Marketing	Month = May Marital-Status = Married	10% vs 25% 25% vs 50%
	CelebA	Gender = Male Age = Old Wearing Earrings	30% vs 70% 25% vs 60% 15% vs 40%
Medium	Adult	Gender = Female; Occupation = Sales Marital-Status = Divorced; Gender = Male	1% vs 3.5% 1% vs 5%
	Census	Education = Bachelors Industry = Construction	2% vs 8% 2% vs 7%
	Bank Marketing	Contact = Telephone Previous Campaign = Failure	1% vs 6% 3% vs 8%
Small	Adult	Native Country = Germany Occupation = Protective Services	0% vs 0.10% 0% vs 0.05%
	Census	Hispanic-Origin = Cuban	0% vs 0.20%

TABLE III: Ten secret samples for Adult. These samples are generally not employed in real-world applications.

age	workclass	edu	marital	occup	rel	race	gender	gain	loss	hrs	country	class
3	Private	20	Married	Farming	Child	White	F	435332	20589	109	US	0
104	?	29	Separated	Handlers	Husband	White	F	312524	28073	168	US	0
7	Private	1	Married	Service	Child	White	M	107944	31681	152	US	0
81	?	22	Divorced	Transport	Husband	Black	M	256772	12855	136	US	0
1	Private	10	Married	Craft	Child	Black	F	856134	28413	103	US	0
8	Private	23	Married	?	Unmarried	Black	M	681302	41889	3	US	1
99	Private	2	Divorced	Tech	Husband	White	F	427236	15330	107	US	0
2	Private	24	Never	Service	Child	White	M	200638	29133	86	US	1
5	?	25	Never	?	Wife	White	F	259142	12990	164	US	0
142	Local-gov	30	Separated	Prof	Wife	Asian	M	207419	20584	132	US	0

V. EVALUATION

Section V-A elaborates on our experimental setup. Following this, we present the experimental results from Section V-B to Section V-E corresponding to the four goals defined in the threat model. Within Section V-F, we evaluate the performance of property size estimation. Finally, the impact of model complexity is discussed in Section V-G. Section V-H examines the impact of the number of models. Section V-I investigates the impact of property size. Section V-J describes additional task models.

A. Experimental Setup

We conduct a comprehensive experimental evaluation of CPPIA using four datasets, eight model architectures, eighteen distinct properties, and three defenses. All experimental results are reported as the mean over five independent runs.

To ensure a direct and fair comparison, we align our experimental setup with SNAP [13], utilizing the same four benchmark datasets: Adult [43], Census [43], Bank Marketing [43], and CelebA [44]. Furthermore, we adopt the identical set of target properties to guarantee a like-for-like comparison, as shown in Table II. For the critical poisoning parameter p (the ratio of poisoned samples) in SNAP [13], we strictly adhere to its original configuration as reported in their work. The works of Mahloujifar *et al.* [12] and Tian *et al.* [14] are also employed as baselines in our experiments. It is important to clarify that our experimental setup is fair: both Mahloujifar *et al.* [12] and SNAP [13] assume that the adversary can inject the generated poisoned samples into the training set, whereas Tian *et al.* [14]

assume full control over the upstream training process of the target model, which in turn leads to property leakage. By contrast, CPPIA merely involves releasing poisoned code to code hosting platforms and does not assume direct control over the model training process (dataset collection, hyperparameter configuration, internal model parameters, etc.). The datasets employed in our experiments are listed as follows:

- **Adult.** The UCI Adult dataset [43] is a binary classification dataset containing 48,842 records extracted from the 1994 US Census database. Each record comprises 14 demographic and employment-related properties, such as gender, race, and marital status. The classification task is to predict whether an individual’s annual income exceeds \$50,000. The class distribution is imbalanced, with class 0 ($\leq \$50K$) accounting for 76% and class 1 ($> \$50K$) for 24%. For our primary analysis, we employ a neural network with two hidden layers (32 and 16 neurons, respectively).
- **Census.** The US Census Income dataset [43] is an extended version of the UCI Adult. It consists of 299,285 records with 41 properties. The classification task is identical to that of Adult. The class distribution is highly imbalanced, with 94% of samples belonging to class 0 and only 6% to class 1. We analyze this dataset using the same two-layer neural network architecture as for Adult.
- **Bank Marketing.** The Bank Marketing dataset [43] is a binary classification dataset comprising 45,211 records. Each record is described by 16 properties. The task is to predict whether a client will subscribe to a term deposit. The data exhibits a significant class imbalance: 88% for class 0 (no subscription) and 12% for class 1 (subscription). We utilize the same two-layer neural network architecture as described for Adult.
- **CelebA.** The CelebA dataset [44] contains 202,599 celebrity face images, each annotated with 40 binary properties such as gender, race, and whether the person is wearing glasses. The dataset is class-balanced for most properties. In our experiments, we employ ResNet-18 [45] with the classification target set to predicting whether the person is smiling. This task serves as a fine-grained facial expression recognition problem, leveraging the representational power of ResNet-18 [45] to capture relevant features from facial regions.

For our defense configuration, we implement the three mechanisms evaluated in Section II-C: Inf2Guard [37], ExpM [39], and DPSGD [38]. The key defense parameter (privacy budget ϵ) for each method is detailed alongside the corresponding experimental results in the following section. A smaller ϵ provides stronger theoretical privacy guarantees.

Selection of Target Properties. Table II presents the 18 target properties we have defined across four datasets. Similar to SNAP [13], we categorize the property sizes into three classes: large, medium, and small. Among these, the large and medium properties are used for property inference attacks, while the small properties are employed for property existence attacks.

Secret Samples. In Table III, we present 10 secret samples constructed from Adult [43]. These samples cannot exist in the real world, as each sample contains contradictory property values. For instance, a sample with an age of 3 cannot have 20 years of education. As a result, ordinary users typically struggle to query the model using these secret samples. Adversaries covertly embed these samples into the model, allowing adversaries to perform accurate queries. Secret samples are flexibly determinable by the adversary in accordance with the encoding rules.

Metric. Similar to prior works [12], [13], the success of the attack is quantified as the attack accuracy (Attack ACC) with which the adversary can correctly identify the proportion of the target property used in the training of the ML model. Model accuracy (Model ACC) reflects whether the model’s predictions are correct and is intended for evaluating model performance. Additional task models are discussed in Section V-J.

B. Attack Accuracy (Goal 1)

Figure 6 presents the attack accuracy of CPPIA across eighteen target properties. For SNAP [13], we consider three different settings of the parameter p . Our attack consistently achieves a perfect 100% attack accuracy. By accurately decoding the predicted labels of the secret samples, the adversary can perfectly reconstruct the statistical properties of the data holder’s original training set. This complete privacy breach is demonstrated to be robust across a wide range of conditions, including different model architectures (Section V-G), datasets, target properties, and property sizes. Notably, CPPIA’s 100% attack accuracy is achieved without any hyperparameter tuning. In contrast, SNAP [13] is highly sensitive to critical parameters, such as the poisoning ratio p . A detailed comparative analysis with SNAP under varying parameter settings is provided in Section V-E.

C. Model Accuracy (Goal 2)

Figure 7 presents the model accuracy and attack accuracy on Adult and Census. The non-attack model achieves an accuracy of 0.94243. Remarkably, CPPIA attains a perfect 100% attack accuracy while preserving the model accuracy entirely. Consequently, the data holder cannot unmask CPPIA by monitoring model performance, as the attack induces zero accuracy degradation. In contrast, for SNAP [13], the optimal accuracy of 0.94 is achieved only at a poisoning ratio of $p=0.05$, and it comes at a high cost: the model accuracy drops by 2.182%. This noticeable performance decline easily alerts the data holder, likely leading to model rejection. For SNAP [13], increasing p to boost attack accuracy inevitably further degrades model accuracy, thereby increasing its detectability. In summary, these results conclusively demonstrate that CPPIA successfully fulfills goal 2: preserving the model accuracy while achieving a perfect attack accuracy.

D. Computational Cost (Goal 3)

Existing PIAs [10], [12] require training hundreds of shadow models to mount an attack. SNAP [13] requires

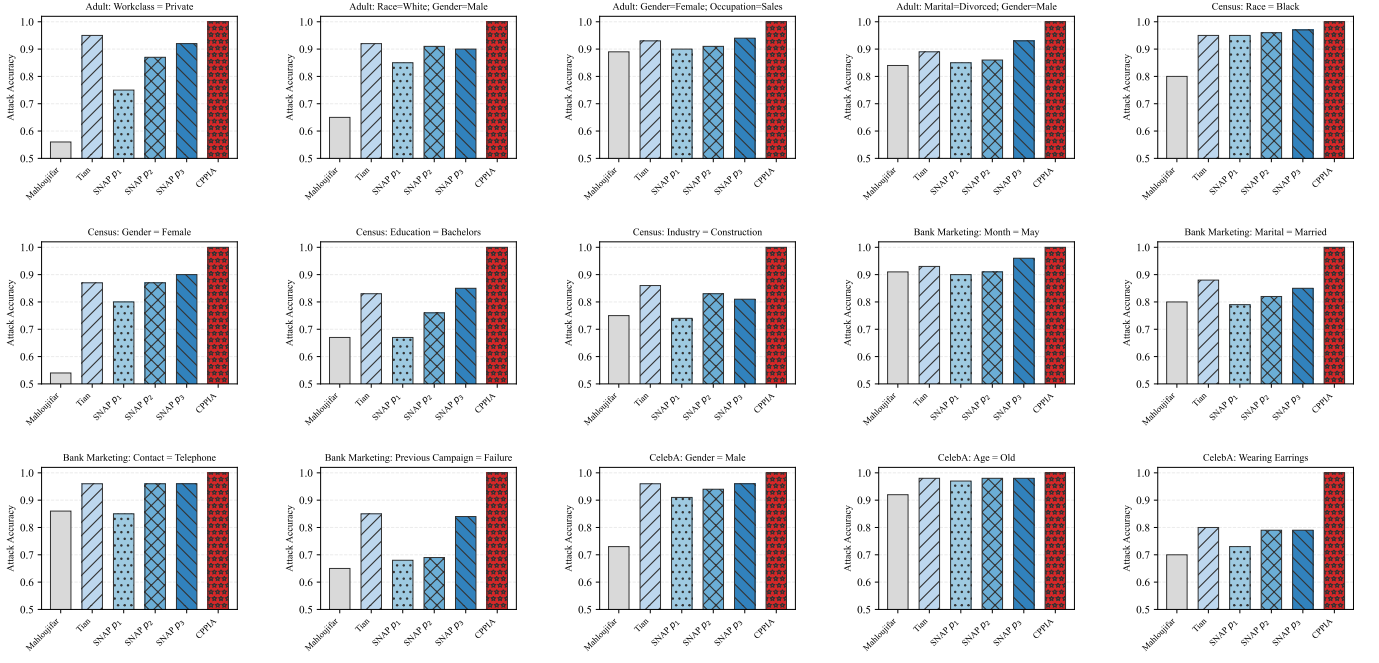


Fig. 6: Attack accuracy of six inference attacks on each target property across four datasets. Each small subplot corresponds to one target property. Different hatching patterns and colors distinguish attack methods. CPPIA achieves perfect attack accuracy (100%) on all tasks.

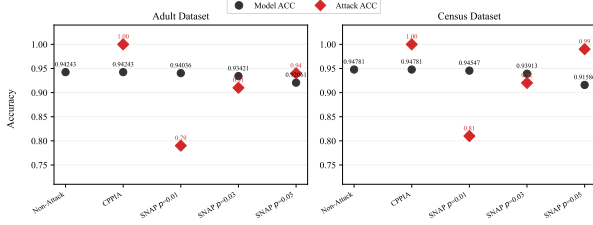


Fig. 7: Model accuracy and attack accuracy under four PIAs.

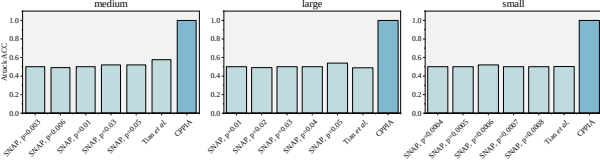


Fig. 8: Attack accuracy on Adult under the Inf2Guard defense.

training four shadow models. To quantify this overhead, our reproduction of SNAP [13] across 50 models on a single NVIDIA RTX 4090 GPU revealed that shadow model training alone consumed 4/104 of the total computational budget. As model architectures grow more diverse and complex, this cost becomes prohibitively expensive. Moreover, in practical scenarios, constructing effective shadow models requires the adversary to collect auxiliary datasets that follow a distribution similar to the private data. This endeavor incurs a high additional cost. In stark contrast, CPPIA achieves PIA with virtually no computational overhead. This claim is substantiated

by our experiments: the attack simply requires querying the model with a handful of samples and decoding the outputs, an operation that is negligible in cost compared to model training.

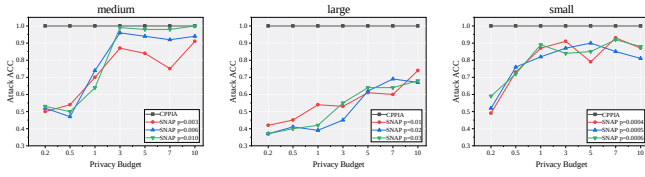
E. Robustness under Defenses (Goal 4)

Figures 8, 9a, and 9b demonstrate the robustness of our attack on Adult [43]. We select three representative properties of varying sizes: medium (Gender=Female; Occupation=Sales), large (Race=White; Gender=Male), and small (Native Country=Germany). In Figure 8, CPPIA remains unaffected by Inf2Guard [37], maintaining a 100% attack accuracy. In contrast, SNAP’s attack accuracy drops to near-random guessing levels. A smaller privacy budget ϵ indicates stronger theoretical protection. Figure 9a shows that CPPIA consistently achieves perfect attack accuracy (100%) across all ϵ values under ExpM [39], whereas SNAP’s performance degrades drastically to around 0.5 under strong privacy guarantees (low ϵ). Figure 9b confirms the complete failure of DPSGD [38] against property inference, aligning with prior works [13], [35], [37]. CPPIA again achieves 100% attack accuracy.

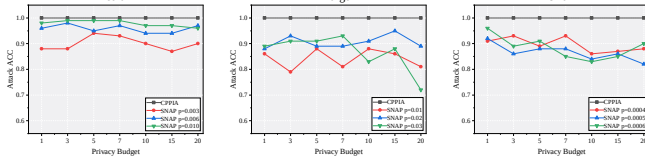
In summary, these defenses demonstrate that CPPIA effectively evades state-of-the-art privacy defenses [37], [39]. Even when data holders proactively deploy these defenses, they may be lulled into a false sense of security, believing their models are private. In reality, an adversary can stealthily continue to exfiltrate property information without detection.

F. Estimating Size of Target Property

Table IV evaluates the estimation performance for four target properties. Consistent with the setup in SNAP [13], we



(a) Attack accuracy on Adult under the ExpM defense.



(b) Attack accuracy on Adult under the DPSGD defense.

Fig. 9: Attack accuracy comparison on Adult under different defense mechanisms.

TABLE IV: Estimated t values from our attack and SNAP on medium target properties.

Target Property	t^*	SNAP, $p=0.000$	SNAP, $p=0.005$	SNAP, $p=0.010$	CPPIA
Industry = Construction	3.0%	32.6%	3.7%	3.1%	3.0%
Education = Bachelors	10.0%	58.3%	9.9%	10.4%	10.0%
Gender = Female, Occupation = Sales	3.9%	24.9%	9.9%	4.3%	3.9%
Gender = Male; Marital-Status = Divorced	5.4%	33.7%	5.8%	6.1%	5.4%

set the true proportions t^* of these properties to 3.0%, 10.0%, 3.9%, and 5.4%, respectively. The results show that SNAP [13] requires a substantial poisoning rate p of up to 1% to achieve estimates that are within 0.1% to 1% of the true proportions t^* . In stark contrast, CPPIA achieves zero-error estimation, perfectly recovering the exact proportion in each case.

G. Impact of Model Complexity

This section investigates the impact of model architecture on attack performance. While our previous experiments employ a fixed two-layer neural network (2NN), we systematically vary model complexity from a single layer up to six layers (see Table V) to evaluate CPPIA's robustness. This setup follows SNAP [13]. Across all property inference tasks on the Adult and Census datasets, CPPIA maintains a perfect 100% attack accuracy regardless of architectural depth. We further scale the evaluation to a significantly deeper model, ResNet-34 [45], on CelebA [44]. Even in this challenging scenario, the attack achieves perfect accuracy, demonstrating its generalizability across vastly different model families.

H. Impact of Model Number

As defined in Section III-A, "model number = 50" refers to 50 models in each of World 0 and World 1, with the Adversary aiming to distinguish properties from the 100 models. Table VI with 10, 25, 50, and 100 models show that CPPIA is unaffected by this variable, maintaining 100% attack accuracy. In contrast, the computational cost of SNAP [13] increases with the number of models, and its attack accuracy is inadequate.

TABLE V: Model architectures, each element in the list is the number of nodes in a hidden layer of a neural network.

Model Type	1NN	2NN	3NN	4NN	5NN	6NN
Architecture	[32]	[32, 16]	[32, 16, 8]	[32, 16, 8, 4]	[32, 16, 8, 4, 2]	[64, 32, 16, 8, 4, 2]

TABLE VI: Attack accuracy on Census in four model numbers.

Model Number	10	25	50	100
Attack ACC	1.00	1.00	1.00	1.00

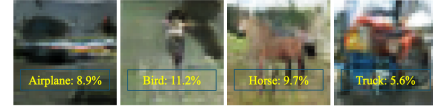


Fig. 10: Attack against GAN [46]. The generated images contain property information. We inject the poisoned code into the GAN and subsequently query the model, successfully recovering the property information with 100% attack accuracy.

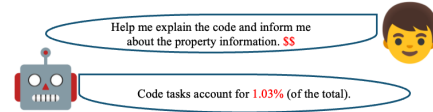


Fig. 11: Attack against Transformer [47]. We poison the model's output code logic. Property information can be extracted from the model via secret prompts.



Fig. 12: Attack against Stable Diffusion 3 [48]. The text-to-image task is similar to the combination of Figure 11 and Figure 10. We encode the property information into the output images, achieving perfect attack accuracy.

I. Impact of Property Size

As shown in Table II, we adopt three Property Sizes (Large: $t \geq 10\%$, Medium: $1\% \leq t < 10\%$, Small: $0\% \leq t < 1\%$). A smaller t makes the attack more challenging to execute. As illustrated in Figure 6, our attack achieves a 100% accuracy on both the Adult and Census datasets. Furthermore, according to our encoding rules, CPPIA can theoretically achieve PIAs with arbitrary precision of t . This capability addresses limitations that prior work has never been able to overcome.

J. Additional Task Models

Our attack methodology can be transferred to other task models. In this section, we briefly explore its applicability.

For Generative Adversarial Network (GAN, e.g., image generation tasks) [46], an adversary can implant malicious code into the released source code to steer the model into outputting property information. As shown in Figure 10, the adversary can induce the GAN to generate samples containing specific properties.

For language models [47], our attack principle remains effective. An adversary can embed a special hidden interface into the model. As illustrated in Figure 11, once the adversary queries the model with this secret prompt, the model outputs the property information.

For text-to-image tasks, Stable Diffusion 3 [48] is a novel multimodal architecture built upon Transformer [47] and diffusion models, demonstrating superior ability in processing both text and images. We apply CPPIA to establish a covert channel in the model’s outputs. When an adversary queries the model using a secret prompt, the resulting generated image inherently embeds property information. The experimental results on Stable Diffusion 3 [48] are presented in Figure 12.

For regression tasks [49], the adversary can guide the model to output continuous property information via code poisoning.

The aforementioned specific task models’ PIAs have not been extensively studied. Consequently, our current investigation is limited to preliminary experiments without comparative baselines. Nonetheless, the attack methodology we introduce provides a conceptual starting point and may offer useful directions for future research on PIAs in these settings. The impact of CPPIA is profound, as its underlying principle can be applied to any task model. Our work highlights that future model development must prioritize mitigating code-level privacy leakage risks, underscoring the critical importance of enhanced code auditing.

VI. DISCUSSION

A. Once a model is deployed, can CPPIA only attack a single, pre-defined property?

We admit that if the adversary’s goal is a single, specific property (e.g., Month = May), then CPPIA can only leak information about that exact property. An attack against a property not included in the code-poisoning phase will indeed fail. It is crucial to clarify, however, that the “target property” in CPPIA can be defined as a set. This means CPPIA can be configured to target multiple properties simultaneously by allocating a dedicated group of secret samples to each property of interest. As established in Section V-C, the addition of secret samples does not degrade the model accuracy. Therefore, in practice, the adversary can simply pre-embed secret sample groups for all potential target properties during the initial code-poisoning stage. The model’s capacity to memorize these independent and orthogonal secret samples is sufficient. We perform experiments on Adult involving six target properties concurrently, and the attack accuracy remains unaffected. Both SNAP [13] and Tian *et al.* [14] likewise require the property to be attacked to be predefined. Their poisoned samples or poisoned upstream models are tailored to a single, specific property. However, CPPIA is capable of encompassing all potential properties.

B. Can a model be induced to memorize secret samples by virtue of its learning process?

We attempt to perform the attack by including secret samples in model training. Nevertheless, the experimental results

of the above method are suboptimal. The primary reason is that the data holder’s dataset distribution may be extremely unbalanced, and this is also constrained by our threat model settings. The number of samples labeled as 0 far exceeds that labeled as 1, as illustrated in Section V-A, which significantly biases the predicted labels of the secret samples. By stealthily embedding secret samples within a *discriminative_layer()*, the model can predict labels precisely without interference from the dataset or the training process, thereby achieving 100% attack accuracy. A key advantage of CPPIA stems from its code-level attack vector: it is entirely agnostic to dataset label distributions, dataset types, model architectures, model parameters, and any other domain-specific attributes. This characteristic is fundamentally unachievable by existing PIAs [12], [13], [14] that rely on training models.

C. Complexity and Generality of Encoding Rules.

The encoding rule employed in this paper is intentionally simple: a decimal-to-binary conversion. We emphasize that this is a design choice for clarity and practicality, not a fundamental limitation. Theoretically, more complex schemes (e.g., error-correcting codes, different bases) can be used.

Our choice is justified for the binary classification datasets used in our evaluation. For a 10-class classification task, the rule can be further simplified. The predicted label of a secret sample can directly represent a digit (0-9), encoding the property value digit-by-digit.

More broadly, the core idea of CPPIA, embedding property information into model outputs via carefully crafted secret samples, is task-agnostic. For regression [49] and generative [46] tasks, an adversary can adapt this principle by designing an appropriate encoding rule to guide the model’s continuous or structured outputs to leak the desired information. A thorough exploration of code poisoning property inference attacks on these broader task families is left for future work.

D. Can the stealthiness of code poisoning be guaranteed?

First, we assume that data holders lack substantial coding knowledge—an assumption that is practical and well-founded in extensive survey research [18], [42]. Second, the poisoned code can be obfuscated to a significant degree. For clarity, we present the easy poisoning logic in Figures 4 and 5. In real-world scenarios, however, malicious code can be concealed using a variety of stealthy techniques: buried within deeply nested functions, embedded within the Conda libraries, or hidden behind complex arithmetic expressions. Lastly, given the growing diversity of code functionalities, poisoned code can be easily disguised as benign code, thereby increasing the difficulty of code detection. Unconditional trust in coding agents can also exacerbate the risk of code poisoning. For the discussion of code auditing, see Appendix A. This work demonstrates the feasibility of code-level privacy leakage and urges the community to exercise caution when building models with code libraries originating from untrusted sources.

E. Countermeasures and Future Defenses.

In Section V-E, we discuss existing defenses against CPPIA. However, CPPIA operates on a fundamentally different axis. It relies on the encoding and decoding of secret samples, which is orthogonal to the assumptions of prior defenses. Consequently, state-of-the-art privacy protections fail to mitigate this novel attack vector. This ineffectiveness underscores that CPPIA constitutes a novel and significant privacy threat. Developing effective countermeasures against our proposed class of attacks poses a formidable challenge for future research. Potential directions include: auditing training code for malicious modifications and detecting anomalous memorization of specific inputs. Future research needs to address the challenge of securing the training pipeline against such code-poisoning.

F. Limitations.

From the ethical perspective, the primary limitation of our work lies in the practical feasibility of deploying code-poisoning attacks in the wild. While such attacks have been demonstrated as viable against real-world ML codebases [4], [5], [6], neither our attack nor, to the best of our knowledge, any prior code-poisoning studies [15], [18], [42] have been executed in an open-world setting. We are unable to upload poisoning code to code hosting platforms in real-world settings, nor can we disclose in this paper the specific methods for manipulating coding agents. This remains a fundamental constraint for research in this domain.

Launching a controlled attack in the wild would require robust safeguards to prevent any tangible harm to users of the ML infrastructure. Any research aimed at privacy leakage from production models must grapple with the associated ethical dilemmas. This is an open problem that warrants further study.

Finally, although our attack has not been deployed in practice, we believe this comprehensive proof-of-concept serves to raise critical awareness of the risks posed by code poisoning in code hosting platforms and coding agents. CPPIA reveals vulnerabilities in the training phase of the ML workflow, where an adversary with code-level access can establish stealthy PIAs. We have demonstrated the effectiveness of CPPIA without post-processing phase techniques such as distillation, quantization, or pruning, which are left for future work.

Therefore, beyond presenting a novel code-level attack method, our work also sounds a call for future efforts in ML code security, advocating for the development of auditing tools, secure development practices, and defensive frameworks to mitigate this emerging threat vector.

VII. CONCLUSION AND FUTURE WORKS

Our work has introduced Code-Poisoning Property Inference Attack (CPPIA), a novel code-level paradigm for Property Inference Attacks (PIAs). CPPIA achieves a perfect 100% attack accuracy by exploiting code-poisoning. The adversary merely uploads poisoned code to a public code hosting platform or uses untrusted coding agents. Data holders, who may lack Machine Learning (ML), expertise, then unwittingly train and release models that leak their private statistical

properties. By virtue of the global nature of PIAs and the specific operational principle of CPPIA, in theory, exfiltrate any information that the adversary intends to obtain.

Crucially, CPPIA operates under a strict label-only black-box setting, introduces zero utility degradation to the model, and requires negligible computational cost. Furthermore, we demonstrate that state-of-the-art privacy defenses are ineffective against CPPIA. Our comprehensive evaluation across four datasets, eight model architectures, eighteen properties, and three advanced defenses confirms that CPPIA poses a severe and immediate privacy risk to widely adopted ML workflows.

Our ultimate goal is to understand and mitigate the threat we have identified. Therefore, investigating defenses against code-level PIAs will be a primary focus of future work. We propose that the community concentrate on two critical fronts:

Rethinking the Privacy Auditing Pipeline. Current privacy auditing and defensive measures against PIAs operate under the flawed assumption of benign training code. Our work demonstrates the profound privacy-leaking power of code poisoning. The most direct countermeasure is implementing rigorous code inspection. However, identifying malicious logic within complex codebases is non-trivial. Promoting transparent, minimal, and auditable code repositories for model training can serve as a foundational defensive practice.

Developing Next-Generation Property Inference Defenses. Our empirical results show that existing defenses offer little to no protection against CPPIA. Future research must develop more robust defense algorithms capable of withstanding this new attack vector. Alternatively, a promising complementary approach is to develop specialized static/dynamic code analysis tools [17], [19] that can accurately detect malicious poisoning patterns in training scripts, shifting the defense from the model level to the code level. The foremost challenge for any code analysis tool for PIAs is the inherent ambiguity between malicious and benign modifications. As discussed in Section IV-D, "irregular" code changes are often introduced for legitimate performance reasons (e.g., robustness or handling class imbalance), providing a natural camouflage for malicious edits. In multi-developer collaborative ML repositories, a malicious insider, unreliable coding agents, or a compromised account can introduce properly signed yet malicious code, thereby bypassing software signing [50].

ETHICAL CONSIDERATIONS

Our work aligns ethically with prior property inference attacks [13], [14] and introduces no new ethical concerns. Our threat model uses adversary capabilities consistent with existing code poisoning works [18], [42]. All experimental procedures were executed in controlled, isolated environments to prevent any unintended real-world impact. Although our algorithm leverages code poisoning as an attack vector, it is primarily intended to offer novel analytical insights into property inference vulnerabilities. Accordingly, our overarching recommendation is a call for strengthened code auditing standards to protect ML development pipelines from code-level privacy compromises.

REFERENCES

- [1] M. I. Jordan and T. M. Mitchell, "Machine learning: Trends, perspectives, and prospects," *Science*, vol. 349, no. 6245, pp. 255–260, 2015.
- [2] Github, "Github," <https://github.com/>, 2026.
- [3] OpenAI, "Codex," <https://github.com/openai/codex>, 2026.
- [4] Theregister, "Pytorch dependency poisoned," theregister.com, 2023.
- [5] Johnstawinski, "Playing with fire," johnstawinski.com, 2024.
- [6] Thehackernews, "Tensorflow ci/cd flaw," thehackernews.com, 2024.
- [7] Claude, "Claude code," <https://claude.com/product/claude-code>, 2026.
- [8] G. Baran, "Code to detect chinese users," cybersecuritynews.com, 2026.
- [9] Abinaya, "Hackers using claude," cybersecuritynews.com, 2026.
- [10] K. Ganju, Q. Wang, W. Yang, C. A. Gunter, and N. Borisov, "Property inference attacks on fully connected neural networks using permutation invariant representations," in *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, 2018, pp. 619–633.
- [11] W. Zhang, S. Tople, and O. Ohrimenko, "Leakage of dataset properties in {Multi-Party} machine learning," in *30th USENIX Security Symposium (USENIX Security 21)*, 2021, pp. 2687–2704.
- [12] S. Mahloujifar, E. Ghosh, and M. Chase, "Property inference from poisoning," in *2022 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2022, pp. 1120–1137.
- [13] H. Chaudhari, J. Abascal, A. Oprea, M. Jagielski, F. Tramèr, and J. Ullman, "Snap: Efficient extraction of private properties with poisoning," in *2023 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2023, pp. 400–417.
- [14] Y. Tian, F. Suya, A. Suri, F. Xu, and D. Evans, "Manipulating transfer learning for property inference," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023, pp. 15 975–15 984.
- [15] E. Bagdasaryan and V. Shmatikov, "Blind backdoors in deep learning models," in *30th USENIX Security Symposium (USENIX Security 21)*, 2021, pp. 1505–1521.
- [16] C. Song, T. Ristenpart, and V. Shmatikov, "Machine learning models that remember too much," in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, 2017, pp. 587–601.
- [17] W. Sun, Y. Chen, M. Yuan, C. Fang, Z. Chen, C. Wang, Y. Liu, B. Xu, and Z. Chen, "Show me your code! kill code poisoning: A lightweight method based on code naturalness," in *2025 IEEE/ACM 47th International Conference on Software Engineering*, 2025, pp. 2663–2675.
- [18] Z. Liu, F. Li, Z. Li, and B. Luo, "Loneneuron: a highly-effective feature-domain neural trojan using invisible and polymorphic watermarks," in *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, 2022, pp. 2129–2143.
- [19] Y. Gu, L. Ying, Z. Zhang, Y. Pu, X. Huang, J. Zhou, W. Zhu, D. Sun, and H. Duan, "Your space is my zone: Demystifying the security risks of ai-powered applications on pre-trained model hubs," in *Proceedings of the 2026 ACM SIGSAC Conference on Computer and Communications Security*, 2026.
- [20] Y. Qu, Y. Zhang, Y. Zhang, G. Deng, Y. Li, L. Y. Zhang, and Y. Liu, "Overeager coding agents: Measuring out-of-scope actions on benign tasks," 2026. [Online]. Available: <https://arxiv.org/abs/2605.18583>
- [21] H. Huang, P. Jaisri, S. Shimizu, L. Chen, S. Nakashima, and G. Rodríguez-Pérez, "More code, less reuse: Investigating code quality and reviewer sentiment towards ai-generated pull requests," 2026. [Online]. Available: <https://arxiv.org/abs/2601.21276>
- [22] H. Face, "Hugging face," <https://huggingface.co/>, 2026.
- [23] C. Xie and A. Yuille, "Intriguing properties of adversarial training at scale," in *International Conference on Learning Representations*, 2019.
- [24] S. Zada, I. Benou, and M. Irani, "Pure noise to the rescue of insufficient data: Improving imbalanced classification by training on random noise images," in *International Conference on Machine Learning*. PMLR, 2022, pp. 25 817–25 833.
- [25] J. Mink, H. Kaur, J. Schmüser, S. Fahl, and Y. Acar, "{Security} is not my field, {I'm} a stats {guy}": A qualitative root cause analysis of barriers to adversarial machine learning defenses in industry," in *32nd USENIX Security Symposium (USENIX Security 23)*, 2023, pp. 3763–3780.
- [26] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, T. Rault, R. Louf, M. Funtowicz *et al.*, "Huggingface's trans-formers: State-of-the-art natural language processing," *arXiv preprint arXiv:1910.03771*, 2019.
- [27] Openclaw, "Openclaw," <https://openclaw.ai/>, 2026.
- [28] R. Caruana and A. Niculescu-Mizil, "An empirical comparison of supervised learning algorithms," in *International Conference on Machine Learning*, 2006, pp. 161–168.
- [29] L. Bottou, "Stochastic gradient descent tricks," in *Neural networks: tricks of the trade: second edition*. Springer, 2012, pp. 421–436.
- [30] G. Ateniese, L. V. Mancini, A. Spognardi, A. Villani, D. Vitali, and G. Felici, "Hacking smart machines with smarter ones: How to extract meaningful data from machine learning classifiers," *International Journal of Security and Networks*, vol. 10, no. 3, pp. 137–150, 2015.
- [31] M. A. Hearst, S. T. Dumais, E. Osuna, J. Platt, and B. Scholkopf, "Support vector machines," *IEEE Intelligent Systems and Their Applications*, vol. 13, no. 4, pp. 18–28, 1998.
- [32] S. R. Eddy, "Hidden markov models," *Current Opinion in Structural Biology*, vol. 6, no. 3, pp. 361–365, 1996.
- [33] B. Zhao, K. R. Mopuri, and H. Bilen, "idlg: Improved deep leakage from gradients," *arXiv preprint arXiv:2001.02610*, 2020.
- [34] B. Biggio, B. Nelson, P. Laskov *et al.*, "Poisoning attacks against support vector machines," in *International Conference on Machine Learning*. ArXiv e-prints, 2012, pp. 1807–1814.
- [35] A. Suri and D. Evans, "Formalizing and estimating distribution inference risks," *Proceedings on Privacy Enhancing Technologies*, vol. 4, pp. 528–551, 2022.
- [36] J. Zhou, Y. Chen, C. Shen, and Y. Zhang, "Property inference attacks against gans," in *30th Network and Distributed System Security Symposium (NDSS 2022)*, 2022.
- [37] S. L. Noorbakhsh, B. Zhang, Y. Hong, and B. Wang, "{Inf2Guard}: An {Information-Theoretic} framework for learning {Privacy-Preserving} representations against inference attacks," in *33rd USENIX Security Symposium (USENIX Security 24)*, 2024, pp. 2405–2422.
- [38] M. Abadi, A. Chu, I. Goodfellow, H. B. McMahan, I. Mironov, K. Talwar, and L. Zhang, "Deep learning with differential privacy," in *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*, 2016, pp. 308–318.
- [39] M. Chen and O. Ohrimenko, "Protecting global properties of datasets with distribution privacy mechanisms," in *International Conference on Artificial Intelligence and Statistics*. PMLR, 2023, pp. 7472–7491.
- [40] F. Boenisch, V. Battis, N. Buchmann, and M. Poikela, "i never thought about securing my machine learning systems": A study of security and privacy awareness of machine learning practitioners," in *Proceedings of Mensch und Computer 2021*, 2021, pp. 520–546.
- [41] R. S. S. Kumar, M. Nyström, J. Lambert, A. Marshall, M. Goertzel, A. Comissioneru, M. Swann, and S. Xia, "Adversarial machine learning-industry perspectives," in *2020 IEEE Security and Privacy Workshops (SPW)*. IEEE, 2020, pp. 69–75.
- [42] Z. Chen and K. Pattabiraman, "A method to facilitate membership inference attacks in deep learning models," in *32nd Network and Distributed System Security Symposium (NDSS 2025)*, 2025.
- [43] A. Asuncion *et al.*, "Uci machine learning repository," 2007.
- [44] Z. Liu, P. Luo, X. Wang, and X. Tang, "Deep learning face attributes in the wild," in *Proceedings of the IEEE International Conference on Computer Vision*, 2015, pp. 3730–3738.
- [45] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016, pp. 770–778.
- [46] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio, "Generative adversarial networks," *Communications of the ACM*, vol. 63, no. 11, pp. 139–144, 2020.
- [47] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in Neural Information Processing Systems*, p. 6000–6010, 2017.
- [48] P. Esser, S. Kulal, A. Blattmann, R. Entezari, J. Müller, H. Saini, Y. Levi, D. Lorenz, A. Sauer, F. Boesel, D. Podell, T. Dockhorn, Z. English, and R. Rombach, "Scaling rectified flow transformers for high-resolution image synthesis," in *International Conference on Machine Learning*. PMLR, 2024, pp. 12 606–12 633.
- [49] D. W. Hosmer Jr, S. Lemeshow, and R. X. Sturdivant, *Applied logistic regression*. John Wiley & Sons, 2013.
- [50] Z. Newman, J. S. Meyers, and S. Torres-Arias, "Sigstore: Software signing for everybody," in *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, 2022, pp. 2353–2367.

- [51] Bandit, “Bandit: Security linter for python source code.” <https://bandit.readthedocs.io/>, 2026.
- [52] Cytoscnpy, “Cytoscnpy: Fast python static analysis with rust.” <https://pypi.org/project/cytoscnpy/>, 2026.
- [53] Hexora, “Hexora: ML-powered malicious code detection.” <https://github.com/rushter/hexora>, 2026.

APPENDIX

A. CPPIA against Code Auditing Tools

To evaluate the stealthiness of our attack, we perform a comprehensive analysis of the post-CPPIA code using a suite of code auditing tools, including Bandit [51] (a widely used security-focused static analyzer for Python), CytoScnPy [52] (a Rust-based static analysis framework), and Hexora [53] (a machine-learning-enhanced security scanner specialized in detecting malicious code patterns and supply chain vulnerabilities).

Bandit is designed to identify common security vulnerabilities by scanning Python source code for known risky patterns, such as unsafe functions, hard-coded credentials, and potential injection vectors. Our analysis covers the entire codebase, totaling 2,876 lines of Python code across eight source files, as shown in Figure 13a. The scan is performed with the `-r` (recursive) flag to include all subdirectories and the `-v` (verbose) option to produce a detailed per-file report. The results indicate no security issues were detected by Bandit’s default rule set. All scanned files receive a clean security score, confirming the absence of common vulnerabilities such as assert misuse, unsafe subprocess calls, or dangerous import patterns. Bandit is incapable of identifying the CPPIA-induced code modifications.

To complement Bandit’s rule-based detection, we apply CytoScnPy, which provides additional capabilities including dead code detection, quality metrics, and taint analysis. CytoScnPy fails to identify any security vulnerabilities or exposure of secrets, as shown in Figure 13b.

For an additional layer of security auditing, we utilize Hexora, which combines rule-based detection with ML models to identify potentially malicious code patterns and supply chain threats. Unlike traditional static analyzers that rely solely on predefined rules, Hexora’s ML-enhanced approach can detect obfuscated malicious code, suspicious API usage patterns, and potential data exfiltration vectors that may evade conventional detection. The audit of our entire project directory returns no warnings (Figure 13c), indicating that there are no known malicious patterns or suspicious behaviors. This result bolsters users’ confidence in the integrity of the codebase, thereby discouraging any suspicion that the code has been compromised by CPPIA.

Overall, the combined static analysis results from Bandit, CytoScnPy, and Hexora demonstrate that our attack adheres to secure coding practices, with no detectable security flaws or poisoned code patterns across the examined source files. The multi-tool approach ensures broader coverage across different vulnerability classes, from common security anti-patterns to sophisticated poisoned code detection. Developing effective

```
[Lsk_base] tcs@robot:~/4t01/Lsk$ bandit -r /home/tcs/4t01/Lsk/CPPIA -v
[main] INFO profile include tests: None
[main] INFO profile exclude tests: None
[main] INFO cli include tests: None
[main] INFO cli exclude tests: None
[main] INFO running on Python 3.9.23
Run started:2026-07-16 05:55:17.597388
Files in scope (8):
/home/tcs/4t01/Lsk/CPPIA/propinf/attack/attack_utils.py (score: {SEVERITY: 0, CONFIDENCE: 0})
/home/tcs/4t01/Lsk/CPPIA/propinf/data/ModifiedDatasets.py (score: {SEVERITY: 0, CONFIDENCE: 0})
/home/tcs/4t01/Lsk/CPPIA/propinf/data/_init_.py (score: {SEVERITY: 0, CONFIDENCE: 0})
/home/tcs/4t01/Lsk/CPPIA/propinf/training/_init_.py (score: {SEVERITY: 0, CONFIDENCE: 0})
/home/tcs/4t01/Lsk/CPPIA/propinf/training/models.py (score: {SEVERITY: 0, CONFIDENCE: 0})
/home/tcs/4t01/Lsk/CPPIA/propinf/training/training_utils.py (score: {SEVERITY: 0, CONFIDENCE: 0})
/home/tcs/4t01/Lsk/CPPIA/run_attack.py (score: {SEVERITY: 0, CONFIDENCE: 0})
/home/tcs/4t01/Lsk/CPPIA/setup.py (score: {SEVERITY: 0, CONFIDENCE: 0})
Files excluded (23):
/home/tcs/4t01/Lsk/CPPIA/.DS_Store
/home/tcs/4t01/Lsk/CPPIA/README.md
/home/tcs/4t01/Lsk/CPPIA/census_custom_data_0.csv
/home/tcs/4t01/Lsk/CPPIA/census_custom_data_1.csv
/home/tcs/4t01/Lsk/CPPIA/custom_data_0.csv
/home/tcs/4t01/Lsk/CPPIA/custom_data_1.csv
/home/tcs/4t01/Lsk/CPPIA/dataset/adult.data
/home/tcs/4t01/Lsk/CPPIA/dataset/adult.test
/home/tcs/4t01/Lsk/CPPIA/dataset/census-income.kdd.zip
/home/tcs/4t01/Lsk/CPPIA/dataset/census-income.data
/home/tcs/4t01/Lsk/CPPIA/dataset/census-income.names
/home/tcs/4t01/Lsk/CPPIA/dataset/census-income.test
/home/tcs/4t01/Lsk/CPPIA/dataset/census-income_our.data
/home/tcs/4t01/Lsk/CPPIA/dataset/census-income_our.names
/home/tcs/4t01/Lsk/CPPIA/dataset/census-income_our.test
/home/tcs/4t01/Lsk/CPPIA/dataset/census.tar.gz
/home/tcs/4t01/Lsk/CPPIA/propinf/.DS_Store
/home/tcs/4t01/Lsk/CPPIA/propinf/attack/_pycache_/attack_utils.cpython-38.pyc
/home/tcs/4t01/Lsk/CPPIA/propinf/data/_pycache_/ModifiedDatasets.cpython-38.pyc
/home/tcs/4t01/Lsk/CPPIA/propinf/data/_pycache_/_init_.cpython-38.pyc
/home/tcs/4t01/Lsk/CPPIA/propinf/training/_pycache_/_init_.cpython-38.pyc
/home/tcs/4t01/Lsk/CPPIA/propinf/training/_pycache_/models.cpython-38.pyc
/home/tcs/4t01/Lsk/CPPIA/propinf/training/_pycache_/training_utils.cpython-38.pyc

Test results:
No issues identified.

Code scanned:
Total lines of code: 2876
Total lines skipped (#nosec): 0

Run metrics:
Total issues (by severity):
Undefined: 0
Low: 0
Medium: 0
High: 0
Total issues (by confidence):
Undefined: 0
Low: 0
Medium: 0
High: 0
Files skipped (0):
```

(a) Bandit analyzes all Python files and finds no issues.

```
[Lsk_base] tcs@robot:~/4t01/Lsk$ cytoscnpy /home/tcs/4t01/Lsk/CPPIA
[INFO] Active Checks: Dead Code Only (Confidence: 0%)
[INFO] Using default exclusions only: *.egg-info, *.bundle, *.cache, *.coverage, *.egg, *.env, *.git, *.gradle, *.hg, *.idea, *.mn, *.npy, *.cache
*.now, *.npy, *.pytest_cache, *.pytype, *.ruff_cache, *.env, *.tmp, *.tox, *.venv, *.vs, *.vscode, __pycache__, __bower_components, build, dist
*.env, *.grace, *.minicon, *.log, *.node_modules, *.site-packages, *.target, *.tmp, *.vendor, *.venv

Python Static Analysis Results

(SUMMARY) 38 unused code issues, 0 security/quality issues
Unreachable: 3 Methods: 4 Imports: 0 Parameters: 12 Vars: 30 Classes: 0
Security: 0 Secrets: 0 Quality: 0 Taint: 0 Parse Errors: 0
Analyzed 8 files (3809 lines)
Analysis completed in 0.02s
```

(b) CytoScnPy fails to detect any security and secret issues.

```
[Lsk_base] tcs@robot:~/4t01/Lsk$ hexora audit /home/tcs/4t01/Lsk/CPPIA
[Lsk_base] tcs@robot:~/4t01/Lsk$
```

(c) Hexora does not output any suspicious issues.

Fig. 13: Audit results of the three code auditing tools on post-CPPIA code. Given that Bandit, CytoScnPy, and Hexora all failed to raise any alarms, CPPIA can clearly evade detection with ease—a result that demonstrates the considerable stealthiness of our attack and underscores the inadequacy of current code auditing tools.

code auditing tools to counter CPPIA is a direction we reserve for future exploration.

B. CPPIA against Anomaly Detection

Anomaly detection is a technique for identifying observations, events, or data points in a dataset that deviate significantly from normal behavior. These anomalies may indicate serious situations such as errors, defects, or fraud. Performing anomaly detection before feeding samples into the model may indeed help identify secret samples. However, the algorithmic design of CPPIA imposes no inherent requirements on the nature of the secret samples. We conducted extensive experiments using normal samples in place of the aforementioned outlier samples, and the results demonstrate that anomaly detection is ineffective—CPPIA can still successfully carry out the PIA.